

LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 4: MENGOPERASIKAN DATABASE DAN GENERATE DATA
OTOMATIS DENGAN LARAVEL



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum Modul 4 ini adalah sebagai berikut:

1. Memahami fungsi migration sebagai pencatat riwayat perubahan struktur database pada Laravel.
2. Menghubungkan project Laravel dengan database MySQL melalui file .env.
3. Membuat dan menjalankan migration sehingga tabel mahasiswa terbentuk di dalam database.
4. Membuat model Mahasiswa yang terhubung dengan tabel mahasiswa melalui Eloquent ORM.
5. Menyimpan data yang dikirim dari form ke dalam tabel melalui controller dan menampilkannya kembali pada halaman daftar.
6. Mengisi tabel secara otomatis menggunakan database seeding dengan library Faker.
7. Menganalisis perbedaan antara data yang disimpan dalam array dan data yang disimpan di database, termasuk kendala yang muncul selama pengerjaan.

2. Alat dan Bahan

Praktikum ini dikerjakan menggunakan satu buah laptop dengan prosesor Intel Core i7-13650HX dan sistem operasi Windows 11. Perangkat lunak yang digunakan meliputi Laragon sebagai lingkungan pengembangan web lokal beserta Apache dan MySQL di dalamnya, PHP versi 8.3.30, Composer versi 2.10.3, Laravel Framework versi 13.34.0, MySQL versi 8.4.3 sebagai server database, phpMyAdmin versi 5.2.3 untuk mengelola database, Visual Studio Code sebagai teks editor, Git sebagai version control, dan Google Chrome sebagai browser untuk menguji aplikasi.

3. Langkah-Langkah Praktikum

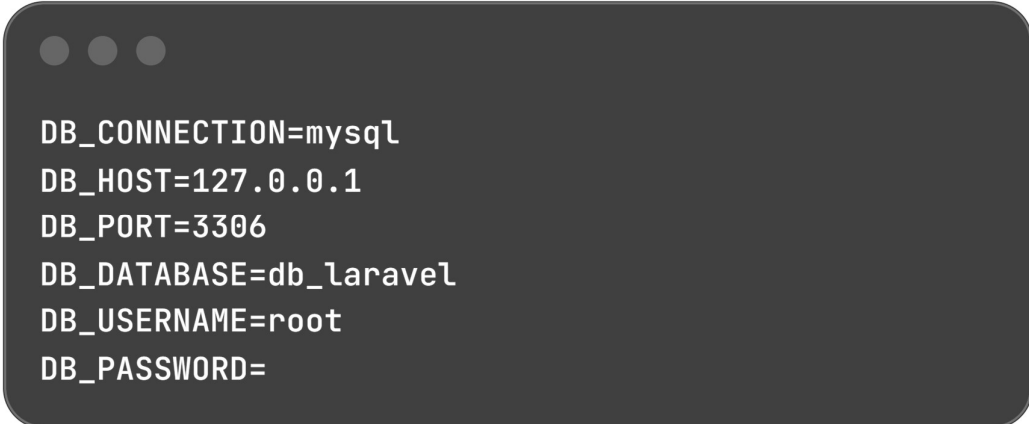
Langkah-langkah praktikum ini diuraikan sesuai materi praktikum 4.4.1 sampai 4.4.14, mulai dari konsep migration, konfigurasi koneksi database, pembuatan migration, model, controller, route, dan view, pengujian aplikasi, sampai pembuatan seeder dengan Faker. Project yang dipakai merupakan lanjutan dari praktikum Modul 3.

3.1 Konsep Migration pada Laravel

Migration adalah fitur Laravel yang mencatat setiap perubahan struktur database dalam bentuk file PHP. Setiap file migration berisi dua method, yaitu `up()` yang dijalankan ketika migration dieksekusi untuk membuat atau mengubah tabel, dan `down()` yang dijalankan ketika rollback untuk membatalkan perubahan tersebut. Karena perubahannya tersimpan sebagai file, struktur database bisa dibangun ulang di komputer lain cukup dengan menjalankan `php artisan migrate`, tanpa perlu membuat tabel satu per satu lewat phpMyAdmin. Cara ini juga memudahkan kerja berkelompok, karena file migration dapat dibagikan ke anggota tim lalu dijalankan pada database masing-masing. Fitur rollback berguna ketika perubahan terakhir ingin dibatalkan.

3.2 Konfigurasi Koneksi Database

Langkah pertama adalah menyalakan Apache dan MySQL dari Laragon, lalu membuat database baru bernama `db_laravel` melalui phpMyAdmin. Setelah database tersedia, project dihubungkan ke database tersebut dengan mengubah bagian koneksi database pada file `.env` di folder utama project.



```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laravel
DB_USERNAME=root
DB_PASSWORD=
```

Nilai `DB_CONNECTION` diisi `mysql` karena server yang dipakai adalah MySQL, `DB_HOST` berisi alamat server (127.0.0.1), `DB_PORT` berisi port MySQL (3306), `DB_DATABASE` berisi nama database (`db_laravel`), `DB_USERNAME` berisi `root`, dan `DB_PASSWORD` dibiarkan kosong karena instalasi Laragon yang saya pakai tidak memberi password pada user `root`. Pada Laravel versi terbaru nilai bawaan `DB_CONNECTION` adalah

sqlite, sehingga baris ini perlu diubah menjadi mysql. Jika perubahan pada .env belum terbaca, Laravel menyediakan perintah `php artisan config:clear` untuk menghapus cache konfigurasi.

3.3 Membuat Migration Tabel Mahasiswa

Tabel mahasiswa dibuat melalui file migration, bukan langsung dari phpMyAdmin. File migration dibuat dengan perintah berikut:

A dark-themed terminal window with three small circles in the top-left corner. It contains a single line of text: `1 php artisan make:migration create_mahasiswa_table`

```
1 php artisan make:migration create_mahasiswa_table
```

Perintah tersebut menghasilkan file baru di folder `database/migrations` dengan nama yang diawali tanggal dan waktu pembuatan, misalnya `2026_10_05_013008_create_mahasiswa_table.php`. Awalan waktu itu dipakai Laravel untuk menentukan urutan eksekusi migration. Isi file tersebut kemudian diganti menjadi seperti berikut:

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('mahasiswa', function (Blueprint $table) {
15             $table->id();
16             $table->string('nama');
17             $table->string('email')->unique();
18             $table->string('jurusan');
19             $table->string('umur');
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      */
27     public function down(): void
28     {
29         Schema::dropIfExists('mahasiswa');
30     }
31 };
32

```

Method up() membuat tabel mahasiswa dengan kolom id sebagai primary key, nama, email yang nilainya harus unik, jurusan, umur, serta kolom timestamps untuk created_at dan updated_at. Method down() menghapus tabel yang sama ketika migration di-rollback. Isi file pada gambar di atas adalah versi akhir, karena kolomnya sempat disesuaikan pada subbab 3.10: kolom nim pada modul 4.4.7 diganti menjadi email dan ditambahkan kolom umur.

Migration dijalankan dengan perintah berikut:



```
php artisan migrate
```

Selain tabel mahasiswa, perintah `php artisan migrate` juga menjalankan migration bawaan Laravel yang membuat tabel `users`, `cache`, dan `jobs`, serta tabel `migrations` sebagai pencatat migration yang sudah dieksekusi. Untuk membatalkan migration terakhir bisa dipakai `php artisan migrate:rollback`. Hasil pengecekan tabel di `phpMyAdmin` ditunjukkan pada Gambar 5.

3.4 Membuat Model Mahasiswa

Model adalah class yang mewakili satu tabel di database. Melalui model, data bisa disimpan dan diambil tanpa menulis perintah SQL. Model dibuat dengan perintah berikut:



```
1 php artisan make:model Mahasiswa
```

Perintah tersebut menghasilkan file `app/Models/Mahasiswa.php`. Isinya kemudian diganti menjadi seperti berikut:

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Mahasiswa extends Model
8  {
9      protected $table = 'mahasiswa';
10
11      protected $fillable = [
12          'nama',
13          'email',
14          'jurusan',
15          'umur',
16      ];
17  }
18
```

Properti \$table diisi 'mahasiswa' karena secara bawaan Laravel menganggap nama tabel adalah bentuk jamak bahasa Inggris dari nama model, sehingga tanpa properti ini model Mahasiswa akan diarahkan ke tabel mahasiswas. Properti \$fillable berisi daftar kolom yang boleh diisi sekaligus lewat satu perintah seperti Mahasiswa::create([...]). Kolom yang tidak tercantum di \$fillable akan ditolak Laravel dengan galat MassAssignmentException.

3.5 Membuat Controller

Controller menjadi penghubung antara view dan model. Controller dibuat dengan perintah berikut:



```
1  php artisan make:controller MahasiswaController
```

File `app/Http/Controllers/MahasiswaController.php` yang terbentuk kemudian diisi dengan kode berikut:

```

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Mahasiswa;
6  use Illuminate\Http\Request;
7
8  class MahasiswaController extends Controller
9  {
10     public function form()
11     {
12         return view('form');
13     }
14
15     public function simpan(Request $request)
16     {
17         Mahasiswa::create([
18             'nama' => $request->nama,
19             'email' => $request->email,
20             'jurusan' => $request->jurusan,
21             'umur' => $request->umur,
22         ]);
23
24         return "Data mahasiswa berhasil disimpan!";
25     }
26
27     public function daftar()
28     {
29         $data = Mahasiswa::all();
30         return view('mahasiswa', ['mahasiswa' => $data]);
31     }
32 }
33

```

Method form() mengembalikan view form, method simpan() menyimpan data yang dikirim dari form menggunakan Mahasiswa::create(), dan method daftar() mengambil seluruh isi tabel dengan Mahasiswa::all() lalu mengirimkannya ke view mahasiswa. Parameter Request \$request berisi data yang dikirim dari form, sehingga nilainya bisa dibaca lewat \$request->nama, \$request->email, \$request->jurusan, dan \$request->umur. Berbeda dengan Modul 3

yang datanya ditulis langsung dalam bentuk array, pada modul ini data benar-benar disimpan ke dan dibaca dari database.

3.6 Mendefinisikan Route

Route menghubungkan alamat URL dengan method pada controller. File routes/web.php diubah menjadi seperti berikut:

```
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\MahasiswaController;
5
6  Route::get('/', function () {
7      return view('welcome');
8  });
9
10 Route::get('/form', [MahasiswaController::class, 'form']);
11 Route::post('/simpan', [MahasiswaController::class, 'simpan']);
12 Route::get('/daftar', [MahasiswaController::class, 'daftar']);
13
```

Route /form dan /daftar memakai Route::get karena hanya menampilkan halaman, sedangkan /simpan memakai Route::post karena menerima data yang dikirim dari form. Baris use App\Http\Controllers\MahasiswaController; di bagian atas file wajib ada agar controller dikenali Laravel. Saya memakai alamat /daftar, bukan /daftar-mahasiswa seperti pada modul 4.4.10, karena nama itu yang sudah dipakai sejak awal pengerjaan.

3.7 Membuat View Form dan Daftar

Ada dua file view yang dibuat di folder resources/views. File pertama adalah form.blade.php untuk menginput data:

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <meta http-equiv="X-UA-Compatible" content="ie=edge">
7      <title>Form Input Mahasiswa</title>
8  </head>
9  <body>
10     <h1>Form Input Mahasiswa</h1>
11     <form action="/simpan" method="POST">
12         @csrf
13         <label for="nama">Nama:</label>
14         <input type="text" name="nama" id="nama" required><br><br>
15
16         <label for="email">Email:</label>
17         <input type="email" name="email" id="email" required><br><br>
18
19
20         <label for="jurusan">Jurusan:</label>
21         <input type="text" name="jurusan" id="jurusan" required><br><br>
22
23         <label for="umur">Umur:</label>
24         <input type="number" name="umur" id="umur" required><br><br>
25
26         <button type="submit">Simpan</button>
27     </form>
28 </body>
29 </html>

```

Directive @csrf menambahkan token perlindungan dari serangan CSRF, atribut action="/simpan" mengarahkan data ke route /simpan, dan method="POST" membuat data dikirim tanpa tampil di URL. Atribut name pada setiap input disamakan dengan nama kolom tabel dan isi \$fillable, sehingga nilainya bisa dibaca controller melalui \$request.

File kedua adalah mahasiswa.blade.php untuk menampilkan data yang sudah tersimpan:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <meta http-equiv="X-UA-Compatible" content="ie=edge">
7     <title>Daftar Mahasiswa</title>
8 </head>
9 <body>
10     <h1>Daftar Mahasiswa</h1>
11     <table border="1" cellpadding="5" cellspacing="0">
12         <thead>
13             <tr>
14                 <th>ID</th>
15                 <th>Nama</th>
16                 <th>Email</th>
17                 <th>Jurusan</th>
18                 <th>Umur</th>
19             </tr>
20         </thead>
21         <tbody>
22             @foreach ($mahasiswa as $mhs)
23                 <tr>
24                     <td>{{ $mhs->id }}</td>
25                     <td>{{ $mhs->nama }}</td>
26                     <td>{{ $mhs->email }}</td>
27                     <td>{{ $mhs->jurusan }}</td>
28                     <td>{{ $mhs->umur }}</td>
29                 </tr>
30             @endforeach
31         </tbody>
32     </table>
33 </body>
34 </html>

```

Variabel \$mahasiswa berisi kumpulan data yang dikirim dari method daftar(). Karena datanya berupa objek Eloquent dan bukan array, setiap kolom dipanggil dengan tanda panah seperti \$mhs->nama. Directive @foreach mengulang baris tabel sebanyak data yang ada di tabel mahasiswa.

3.8 Menjalankan dan Menguji Aplikasi

Server pengembangan dijalankan dengan php artisan serve, lalu alamat <http://127.0.0.1:8000/form> dibuka di browser. Form diisi dengan data mahasiswa, kemudian tombol Simpan ditekan. Setelah dikirim, browser berpindah ke alamat /simpan dan menampilkan teks Data mahasiswa berhasil disimpan! yang berasal dari method simpan().

Halaman <http://127.0.0.1:8000/daftar> kemudian dibuka untuk memastikan data yang baru dikirim ikut tampil bersama data lainnya. Hasil pengujian ditunjukkan pada Gambar 1 sampai Gambar 4.

3.9 Membuat Seeder

Seeder dipakai untuk mengisi tabel dengan data contoh secara otomatis. File seeder dibuat dengan perintah berikut:



```
1 php artisan make:seeder StudentSeeder
```

Perintah tersebut menghasilkan file `database/seeder/StudentSeeder.php`. Di folder yang sama juga terdapat `DatabaseSeeder.php`, yaitu seeder utama bawaan Laravel yang bisa dipakai untuk memanggil beberapa seeder sekaligus dengan `$this->call([...])` di dalam method `run()`. Pada praktikum ini `StudentSeeder` dijalankan langsung dengan opsi `--class`.

3.10 Mengisi Data Dummy dengan Faker

Faker adalah library PHP yang sudah terpasang bersama Laravel dan bisa menghasilkan data palsu seperti nama orang dan alamat email. Isi file `StudentSeeder.php` diganti menjadi seperti berikut:

```

1  <?php
2
3  namespace Database\Seeders;
4
5  use Illuminate\Database\Console\Seeds\WithoutModelEvents;
6  use Illuminate\Database\Seeder;
7  use Illuminate\Support\Facades\DB;
8  use Faker\Factory as Faker;
9
10 class StudentSeeder extends Seeder
11 {
12     /**
13      * Run the database seeds.
14      */
15     public function run(): void
16     {
17         $faker = Faker::create('id_ID');
18
19         for ($i = 0; $i < 20; $i++) {
20             DB::table('mahasiswa')->insert([
21                 'nama' => $faker->name,
22                 'email' => $faker->unique()->safeEmail,
23                 'jurusan' => $faker->randomElement(['Teknik Informatika', 'Sistem
Informasi', 'Teknik Elektro', 'Teknik Mesin', 'Teknik Sipil']),
24                 'umur' => $faker->numberBetween(18, 25),
25                 'created_at' => now(),
26                 'updated_at' => now()
27             ]);
28         }
29     }
30 }
31

```

Faker::create('id_ID') membuat data dengan format Indonesia, perulangan for menjalankan pengisian sebanyak 20 kali, randomElement([...]) memilih jurusan secara acak dari lima pilihan, dan numberBetween(18, 25) menghasilkan umur acak. Data dimasukkan ke tabel lewat DB::table('mahasiswa')->insert([...]) yang memakai Query Builder, sedangkan kolom created_at dan updated_at diisi dengan now().

Sebelum seeder dijalankan, file migration tabel mahasiswa disesuaikan lebih dulu karena seeder mengisi kolom email dan umur, sedangkan migration pada subbab 3.3 awalnya masih memakai kolom nim. Kolom nim diganti menjadi email yang nilainya unik dan ditambahkan kolom umur. Setelah itu tabel dibuat ulang dan seeder dijalankan dengan perintah berikut:

```
1 php artisan migrate:refresh
2 php artisan db:seed --class=StudentSeeder
```

Perintah migrate:refresh menghapus seluruh tabel lalu menjalankan ulang semua migration dari awal, sehingga data lama ikut terhapus. Perintah db:seed --class=StudentSeeder hanya menjalankan seeder tersebut. Hasilnya, tabel mahasiswa terisi 20 data dummy yang bisa dilihat di phpMyAdmin pada Gambar 6.

4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi yang berjalan melalui php artisan serve pada alamat <http://127.0.0.1:8000> serta tampilan tabel mahasiswa di phpMyAdmin.

Form Input Mahasiswa

Nama:

Email:

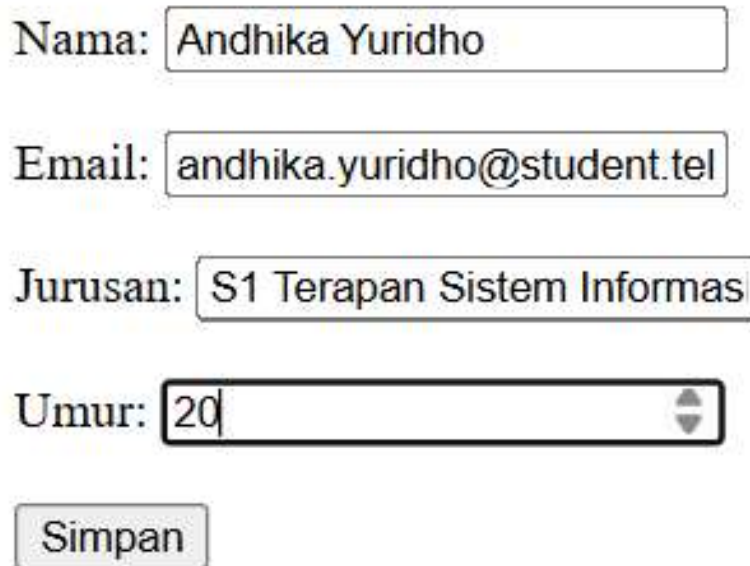
Jurusan:

Umur:

Gambar 1. Halaman form input mahasiswa pada alamat /form

Gambar 1 menampilkan halaman form input mahasiswa dengan empat kolom isian, yaitu nama, email, jurusan, dan umur, serta tombol Simpan di bagian bawah. Halaman ini dirender oleh view form.blade.php melalui method form() pada MahasiswaController.

Form Input Mahasiswa



The image shows a web form titled "Form Input Mahasiswa". It contains four input fields and a submit button. The "Nama:" field contains "Andhika Yuridho". The "Email:" field contains "andhika.yuridho@student.tel". The "Jurusan:" field contains "S1 Terapan Sistem Informas". The "Umur:" field is a spinner box containing "20". Below the fields is a button labeled "Simpan".

Gambar 2. Form yang sudah diisi data mahasiswa

Gambar 2 memperlihatkan form yang sudah diisi dengan nama Andhika Yuridho, email andhika.yuridho@student.telkomuniversity.ac.id, jurusan S1 Terapan Sistem Informasi Kota Cerdas, dan umur 20. Panjang teks di dalam kolom isian memang terpotong secara visual, tetapi nilai yang dikirim tetap utuh karena atribut name pada setiap input sudah sesuai dengan kolom tabel.

Data mahasiswa berhasil disimpan!

Gambar 3. Halaman konfirmasi setelah data disimpan pada alamat /simpan

Gambar 3 menampilkan teks Data mahasiswa berhasil disimpan! yang dikembalikan oleh method simpan() setelah data berhasil dimasukkan ke tabel mahasiswa.

Daftar Mahasiswa

ID	Nama	Email	Jurusan	Umur
1	Sari Anggraini	pranowo.jono@example.com	Teknik Mesin	18
2	Dina Purwanti	winda81@example.org	Teknik Elektro	21
3	Jamal Prasetya	zulaika.tirtayasa@example.org	Teknik Informatika	24
4	Tugiman Jaeman Utama S.Gz	betania12@example.com	Teknik Informatika	24
5	Clara Laksmiwati S.Pd	kalim.nuraini@example.org	Teknik Informatika	22
6	Violet Puspita	idabukke@example.org	Teknik Mesin	25
7	Farah Kuswandari	padma94@example.net	Teknik Mesin	20
8	Dinda Usada	halimah.aurora@example.net	Sistem Informasi	25
9	Paulin Widiastuti	imelani@example.com	Teknik Elektro	23
10	Cici Kasiyah Susanti S.E.	winarsih.fitria@example.net	Sistem Informasi	20
11	Dinda Usada S.IP	arsipatra.sihombing@example.net	Teknik Elektro	22
12	Prasetyo Hidayanto S.I.Kom	lrahmawati@example.org	Teknik Elektro	22
13	Legawa Garda Marbun M.TI.	jhassanah@example.com	Teknik Sipil	23
14	Danu Siregar	hutasoit.vera@example.net	Teknik Elektro	21
15	Gara Vîno Saputra S.IP	nmansur@example.net	Teknik Informatika	23
16	Radit Ismail Kurniawan	gawati.uyainah@example.com	Teknik Elektro	19
17	Niyaga Putra	rusman.sitompul@example.com	Teknik Mesin	22
18	Ira Lestari	efarida@example.net	Teknik Sipil	19
19	Shakila Suryatmi	darman.astuti@example.org	Teknik Sipil	24
20	Perkasa Jindra Simbolon	cindy.marbun@example.org	Teknik Informatika	25
21	Andhika Yuridho	andhika.yuridho@student.telkomuniversity.ac.id	S1 Terapan Sistem Informasi Kota Cerdas	20

Gambar 4. Halaman daftar mahasiswa pada alamat /daftar

Gambar 4 menampilkan 21 baris data pada tabel, yaitu 20 data hasil seeder dan satu data yang saya input sendiri melalui form. Data pada baris terakhir merupakan data yang baru dikirim, sehingga urutannya berada di paling bawah. Seluruh baris berasal dari tabel mahasiswa di database, bukan dari array seperti pada Modul 3.

Server: 127.0.0.1 - Database: db_laravel - Table: mahasiswa

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint	utf8mb4_unicode_ci	UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	nama	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	email	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
4	jurusan	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
5	umur	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
6	created_at	timestamp			Yes	NULL			Change Drop More
7	updated_at	timestamp			Yes	NULL			Change Drop More

Print Move columns Normalize

Add 1 column(s) after updated_at Go

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	21	A	No	
Edit Rename Drop	mahasiswa_email_unique	BTREE	Yes	No	email	21	A	No	

Create an index on 1 columns Go

Partitions

No partitioning defined

Partition table

Information

Gambar 5. Struktur tabel mahasiswa pada phpMyAdmin

Gambar 5 memperlihatkan struktur tabel mahasiswa yang terbentuk dari migration. Tabel ini memiliki tujuh kolom, yaitu id bertipe bigint dengan AUTO_INCREMENT dan PRIMARY, nama, email, jurusan, umur yang bertipe varchar(255), serta created_at dan updated_at bertipe timestamp. Pada bagian Indexes terlihat PRIMARY pada kolom id dan mahasiswa_email_unique bertipe BTREE UNIQUE pada kolom email.

	id	nama	email	jurusan	umur	created_at	updated_at
☐	1	Sari Anggraini	pranowo.jono@example.com	Teknik Mesin	18	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	2	Dina Purwanti	winda81@example.org	Teknik Elektro	21	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	3	Jamal Prasetya	zulaika.tirtayasa@example.org	Teknik Informatika	24	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	4	Tugiman	betania12@example.com	Teknik Informatika	24	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	5	Jaeman Utama S Gz Clara	laksmi.wati@example.org	Teknik Informatika	22	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	6	Violet Puspita	idabukke@example.org	Teknik Mesin	25	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	7	Farah Kuswandani	padma94@example.net	Teknik Mesin	20	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	8	Dinda Usada	halimah.aurora@example.net	Sistem Informasi	25	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	9	Paulin Widiastuti	imelani@example.com	Teknik Elektro	23	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	10	Cici Kasiyah Susanti S E	winarsih.fitria@example.net	Sistem Informasi	20	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	11	Dinda Usada S IP	arsipatra.sihombing@example.net	Teknik Elektro	22	2026-10-05 02:53:40	2026-10-05 02:53:40
☐	12	Prasetyo Hidayanto S I Kom Legawa	lrahmawati@example.org	Teknik Elektro	22	2026-10-05 02:53:40	2026-10-05 02:53:40

Gambar 6. Data hasil seeder pada tabel mahasiswa di phpMyAdmin

Gambar 6 menampilkan isi tabel mahasiswa setelah seeder dijalankan. phpMyAdmin melaporkan 21 baris dengan waktu query 0.0005 detik, dan nama serta email yang muncul berbeda-beda karena datanya dibuat acak oleh Faker.

5. Analisis dan Pembahasan

Perbedaan paling terasa antara aplikasi Modul 3 dan Modul 4 ada pada masa hidup datanya. Pada Modul 3, data mahasiswa ditulis sebagai array di dalam controller, sehingga data yang dikirim lewat form hilang begitu halaman ditutup atau server dijalankan ulang. Setelah memakai database, data disimpan ke tabel mahasiswa dan tetap ada sampai dihapus, sehingga halaman daftar bisa menampilkan seluruh riwayat data yang pernah dikirim.

Struktur tabel mahasiswa dipilih menyesuaikan jenis data yang disimpan. Kolom id dibuat dengan \$table->id() agar setiap baris punya primary key berupa angka yang bertambah otomatis. Kolom nama, email, dan jurusan memakai tipe string karena isinya teks pendek, sedangkan email diberi unique() supaya tidak ada dua mahasiswa dengan alamat email yang sama. Kolom timestamps() menambahkan created_at dan updated_at yang diisi otomatis oleh Laravel, sehingga waktu penyimpanan data tidak perlu dicatat manual. Kolom umur pada file

migration mengikuti modul, yaitu bertipe string, walaupun sebenarnya lebih tepat memakai `$table->integer('umur')` karena isinya angka.

Migration dan seeder punya peran yang berbeda meskipun sama-sama dijalankan lewat artisan. Migration mengurus struktur tabel, misalnya menambah kolom atau mengubah tipe data, dan perubahannya bertahan sampai ada migration baru yang mengubahnya. Seeder mengurus isi tabel, dan datanya bisa dibuat ulang kapan saja dengan menjalankan ulang perintahnya. Karena perannya berbeda, keduanya sering dipakai berurutan: migration dulu untuk membentuk tabel, baru seeder untuk mengisinya dengan data uji.

Kendala yang muncul selama pengerjaan tercatat di file `storage/logs/laravel.log`. Pertama, muncul galat `Target class [app\Http\Controllers\MahasiswaController] does not exist` sebanyak 15 kali. Penyebabnya adalah penulisan namespace pada baris `use` di `routes/web.php` dan pada `use` di dalam controller yang diawali huruf kecil, yaitu `app\Http\Controllers` dan `app\Models`. Nama namespace di PHP bersifat case-sensitive, sehingga App harus ditulis dengan huruf kapital. Setelah penulisan diubah menjadi `App\Http\Controllers\MahasiswaController` dan `App\Models\Mahasiswa`, galatnya hilang. Kedua, saat form dikirim dua kali dengan alamat email yang sama, muncul galat `SQLSTATE[23000] Duplicate entry pada key mahasiswa_email_unique`. Galat ini berasal dari aturan `unique()` pada kolom email, dan teratasi setelah form diisi dengan alamat email yang berbeda.

Ada satu penyesuaian yang saya lakukan terhadap modul. Modul 4.4.10 memakai alamat `/daftar-mahasiswa` untuk menampilkan daftar, sedangkan project saya memakai alamat `/daftar`. Saya membiarkan nama tersebut karena route, view, dan pengujian sudah memakai alamat itu, dan fungsinya tidak berbeda.

6. Kesimpulan

Berdasarkan praktikum Modul 4 ini, saya memahami migration sebagai pencatat riwayat struktur database yang membuat tabel bisa dibentuk ulang cukup dengan satu perintah. Koneksi database berhasil diatur melalui file `.env` dengan `DB_CONNECTION` `mysql` dan database `db_laravel`, lalu tabel mahasiswa terbentuk dari migration dengan kolom `id`, `nama`, `email`, `jurusan`, `umur`, `created_at`, dan `updated_at`. Model Mahasiswa berhasil dihubungkan ke tabel tersebut melalui properti `$table` dan `$fillable`, sehingga penyimpanan data bisa dilakukan dengan `Mahasiswa::create()` dan pengambilan data dengan `Mahasiswa::all()`.

Alur form sampai database juga berjalan sesuai rancangan. Data yang dikirim dari form.blade.php diterima method `simpan()` pada MahasiswaController melalui route POST `/simpan`, disimpan ke tabel mahasiswa, lalu ditampilkan kembali oleh method `daftar()` pada halaman `/daftar`. Seeder dengan Faker berhasil mengisi tabel dengan 20 data dummy berbahasa Indonesia, dan hasilnya bisa dilihat di phpMyAdmin maupun di halaman daftar. Dari praktikum ini terlihat bahwa data yang disimpan di database bertahan jauh lebih lama dibandingkan data array, dan pembagian tugas antara migration dan seeder membuat pengelolaan database lebih rapi.

7. Link Repository Github

<https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/pabw-4/praktikum4>